

•
P U L S E
•

Codebase Setup & Firepit Packaging

A practical guide to creating a maintainable front-end codebase, preparing it for single-file hosting, and releasing it through Firepit.

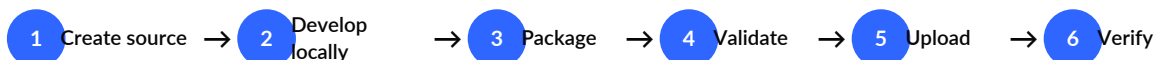
AUDIENCE	SCOPE	DELIVERABLE
Developers, maintainers, and future handoff personnel	Repository setup, local development, packaging, validation, deployment, and rollback	A repeatable starter pattern and release procedure

SOURCE CAN BE MANY FILES. RELEASE IS ONE FILE.

01 / START WITH THE END STATE

Build for maintainability. Package for Firepit.

Firepit is the hosting surface, not the build system. It receives a flat HTML artifact and executes the inline code and styles it contains. That means the development project may be organized across many files, but the release must be one self-contained HTML file.



CORE RULE

Source can be modular. The Firepit release artifact must be one self-contained HTML file.

Do not ship the source folder, separate scripts, separate stylesheets, or runtime CDN dependencies.

FIREPIT DOES	FIREPIT DOES NOT
• Render the supplied HTML inside SharePoint.	• Resolve Node packages or source imports at runtime.
• Execute inline scripts and apply inline styles.	• Bundle files, rewrite paths, or copy images for you.
• Run on the SharePoint origin and inherit the user session.	• Replace local testing, package validation, or release control.

02 / CREATE THE CODEBASE

Start with a predictable project skeleton.

The current PULSE repository uses a modular browser structure. The same separation also maps cleanly to a modern React mental model: shell, routes, state, services, feature modules, assets, and a build output.

Recommended starter structure

```
my-app/
|-- index.html
|-- assets/
|   |-- css/style.css
|   |-- images/
|   |-- js/
|       |-- app-config.js
|       |-- app.js
|       |-- data.js
|       |-- sharepoint-adapter.js
|       |-- sharepoint-repo.js
|       |-- sharepoint-schema.js
|       |-- pages/
|-- vendor/
|-- scripts/
|   |-- build-sharepoint-package.js
|-- releases/
|-- README.md
|-- .gitignore
```

PREREQUISITE	WHY IT IS NEEDED
Node.js	Runs the packaging script and any project tooling.
Git	Tracks the source and identifies the exact commit used for a release.
Modern browser	Runs the local application and developer tools.
Static local server	Serves the source tree without SharePoint. The verified PULSE command is <code>python3 -m http.server 8743</code> .
Approved SharePoint / Firepit access	Required for final identity, permission, write, and host validation.

Version rule: release filenames use semantic versioning: `PULSE-1.0.0.html`. Use MAJOR.MINOR.PATCH - not a calendar date.

03 / ESTABLISH THE APPLICATION SHELL

Make startup and dependency order explicit.

The entry page is the contract between the modular source tree and the packager. Keep it simple, deterministic, and local.

FILE	RESPONSIBILITY	DESIGN RULE
index.html	Entry point and asset order.	Reference local files in the order they must execute.
app-config.js	Non-secret runtime configuration.	Never place credentials, tokens, personal data, or private operational records here.
app.js	Boot, shell, routing, shared helpers.	Keep module startup and route registration centralized.
data.js	Normalized browser state.	Define stable defaults so every screen reads the same object shape.
pages/*.js	One major feature per module.	Own rendering and feature-specific event handling; use shared services for persistence.
style.css	Shared visual system.	Keep responsive rules and theme behavior centralized.

LOCAL ENTRY EXAMPLE

```
<!doctype html>
<html>
<head>
  <link rel="stylesheet" href="assets/css/style.css?devcache=1">
</head>
<body>
  <div id="app"></div>
  <script src="assets/js/app-config.js?devcache=1"></script>
  <script src="assets/js/data.js?devcache=1"></script>
  <script src="assets/js/sharepoint-adapter.js?devcache=1"></script>
  <script src="assets/js/sharepoint-repo.js?devcache=1"></script>
  <script src="assets/js/pages/dashboard.js?devcache=1"></script>
  <script src="assets/js/app.js?devcache=1"></script>
</body>
</html>
```

The ?devcache=1 suffix is local-only. The preferred packager removes it from the release artifact.

04 / ORGANIZE IT LIKE A MODERN FRONT END

Use React concepts without pretending it is React.

A developer familiar with React can understand the codebase quickly when responsibilities are mapped consistently. The current PULSE feature surface is modular browser JavaScript, not a completed React rewrite.

MODERN REACT CONCEPT	CURRENT PULSE EQUIVALENT	PURPOSE
App.tsx	index.html + app.js	Root shell, startup, and route coordination.
Route component	pages/*.js	One renderer and workflow module per major screen.
Context / Redux	data.js	Normalized in-memory application state.
API client	sharepoint-adapter.js	REST transport, identity, digest, and errors.
Repository / service layer	sharepoint-repo.js	Maps browser objects to SharePoint list items.
Schema / migration	sharepoint-schema.js	Defines lists, fields, setup, and repair.
Bundler output	Single HTML package	The release artifact consumed by Firepit.

IF YOU START WITH REACT OR VITE

Compile the application first, then validate that the packaging tool correctly inlines the generated HTML, scripts, styles, fonts, and images.

The current verified builder targets the PULSE source structure. Do not assume compatibility with a new bundler output until the generated package passes the same validation and Firepit tests.

Feature rule: keep screen rendering in feature modules, shared state in one store, external access behind an adapter, and persistent field mapping behind a repository. This keeps changes traceable and testable.

05 / KEEP THE DATA BOUNDARY CLEAN

Trace every change through five layers.

A screen is only the visible end of a data path. New fields and workflows are incomplete until every layer can read, write, normalize, and verify the same information.

1	2	3	4	5
Screen	State	Repository	Adapter	SharePoint
Render and validate input.	Hold normalized browser objects.	Map object fields to list fields.	Send authenticated REST requests.	Persist lists and files.

New-field completion checklist

- 01 Add the field to the schema or configuration.
- 02 Write it in the repository mapper.
- 03 Read and normalize it back into the browser object.
- 04 Add validation, display, and role rules in the UI.
- 05 Run setup if the live SharePoint field must be created or repaired.
- 06 Save, reload, and confirm the value survives a real SharePoint round trip.

Local mode limitation: a browser-only test can prove rendering, routing, validation, and export behavior. It cannot prove SharePoint identity, permissions, schema readiness, REST persistence, document-library access, or Firepit restrictions.

06 / ADD THE PACKAGING TOOL

Firepit hosts. The builder packages.

Place the preferred packaging script at `scripts/build-sharepoint-package.js`. The script turns the modular entry page and local assets into the single flat HTML file Firepit expects.

THE BUILDER DOES	THE BUILDER BLOCKS
• Reads index.html and preserves dependency order.	• External runtime script or stylesheet URLs.
• Inlines linked CSS into style blocks.	• Missing local files or unresolvable asset paths.
• Rewrites CSS url(...) assets into data URIs.	• A package that still depends on the development folder.
• Inlines local scripts and safely escapes closing script text.	• Local cache-busting query strings surviving into release.
• Embeds selected local images.	• Silent packaging when a prohibited dependency is found.
• Adds a base64 WFC-MANIFEST provenance comment.	• Using the heavier iframe-wrapper builder for a direct Firepit release.

USE THE CORRECT TOOL

For direct Firepit hosting, use build-sharepoint-package.js.

The secondary build-forge.js workflow wraps the child page in an encoded iframe template. It is heavier and is not the preferred direct-SharePoint ship path.

07 / USE THE PACKAGING TOOL

Build one versioned release artifact.

Run the build from the application root after local testing and before every upload. The second argument is the exact release path and filename.

PRIMARY BUILD COMMAND

```
cd apps/PULSE
node scripts/build-sharepoint-package.js \
  "releases/PULSE-1.0.0.html"
```

Semantic versioning

VERSION	USE WHEN	EXAMPLE
PATCH	Bug fix or small correction that preserves behavior.	1.0.0 -> 1.0.1
MINOR	Backward-compatible feature or meaningful enhancement.	1.0.1 -> 1.1.0
MAJOR	Breaking data, workflow, interface, or deployment change.	1.1.0 -> 2.0.0

Expected output

ARTIFACT	releases/PULSE-1.0.0.html
CONTENTS	HTML, local JavaScript, CSS, fonts, and selected images contained in one file.
PROVENANCE	A WFC-MANIFEST comment recording generation time and inlined file fingerprints.
SHIP UNIT	Only the generated HTML file - not the source directory.

08 / VALIDATE THE PACKAGE

Do not upload a package you have not inspected.

The builder reduces packaging risk, but it does not replace acceptance testing. Validate the artifact itself, then validate the application in its real hosting boundary.

CHECK	PASS CONDITION
File name	Uses PULSE-MAJOR.MINOR.PATCH.html and matches the intended release.
Single-file boundary	No separate runtime CSS, JavaScript, font, or image files are required.
External dependencies	No prohibited https:// script or stylesheet references remain.
Cache strings	No local ?devcache=N suffix remains.
Manifest	The WFC-MANIFEST provenance comment exists and is readable.
Browser behavior	The package opens, routes render, images display, and no console errors appear.
Regression	Changed and adjacent workflows pass the defined acceptance test.
Rollback	The previous known-good artifact is retained before upload.

LOCAL TEST CAN PROVE	FIREPIT / SHAREPOINT TEST MUST PROVE
Layout, routes, form validation, browser calculations, and export creation.	Current user identity, PULSE roles, list permissions, writes, document access, notifications, and host restrictions.

09 / UPLOAD TO FIREPIT

Release carefully. Rollback quickly.

The exact upload and approval steps depend on the SharePoint site and deployment authority. The operational sequence remains consistent.

- 1 Retain the prior artifact** Copy the last known-good HTML package to a protected rollback location.
- 2 Upload the new file** Place PULSE-1.0.0.html in the approved SharePoint document library that backs Firepit.
- 3 Point Firepit to the file** Use the approved web-part or hosted-page process. Do not improvise around site controls.
- 4 Open as an authorized user** Confirm identity and role resolution in the real SharePoint session.
- 5 Run controlled acceptance tests** Test the changed route, a safe read/write, refresh persistence, files, output, notification, and audit evidence where applicable.
- 6 Record the release** Capture version, source commit, builder, timestamp, uploader, acceptance result, and rollback artifact.

ROLLBACK RULE

Restore first. Investigate second.

If hosted acceptance fails, put the prior known-good artifact back before debugging. Preserve the failed package, browser console, network response, user role, record ID, and before/after state.

10 / QUICK REFERENCE

Starter checklist for every new codebase.

BEFORE CODING

- Create the source tree and releases folder.
- Add the preferred packaging script.
- Keep configuration non-secret.
- Use local copies of required runtime libraries.
- Define the semantic version policy.

BEFORE PACKAGING

- Run local routes and workflows.
- Resolve console errors.
- Confirm every referenced asset exists.
- Remove unintended external dependencies.
- Select the next MAJOR.MINOR.PATCH version.

BEFORE UPLOAD

- Generate one HTML artifact.
- Inspect the WFC-MANIFEST.
- Confirm no ?devcache suffix remains.
- Retain the prior known-good package.
- Write the acceptance test.

AFTER UPLOAD

- Verify identity and role.
- Test a controlled read/write and refresh.
- Verify files, outputs, and notifications.
- Record release evidence.
- Rollback immediately if acceptance fails.

ONE-SENTENCE WORKFLOW

Create a modular browser codebase, package it into PULSE-1.0.0.html, validate the single-file boundary, upload it through the approved Firepit process, and retain evidence plus a rollback artifact.

Source note: current PULSE repository paths, builder behavior, Firepit constraints, and deployment cautions are based on the existing PULSE technical reference. The recommended starter structure is presented as an onboarding pattern and should be verified against the repository before implementation.